1. Data, struktur data, algoritma, dan program

- **Data**: fakta atau nilai mentah, misalnya NIM 22001 atau nama Alya.
- **Struktur data**: cara mengatur dan menyimpan data agar operasi tertentu (akses, cari, tambah, hapus) bisa dilakukan dengan efisien.
- **Algoritma**: langkah-langkah terhingga dan terurut untuk menyelesaikan suatu masalah, misalnya langkah mencari NIM.
- **Program**: implementasi algoritma dan struktur data dalam bahasa pemrograman, sehingga bisa dijalankan komputer.

2. Contoh struktur data

- **Linear**: *stack* (tombol undo pada editor teks) dan *queue* (antrean pelayanan di loket).
- **Non-linear**: *tree* (struktur folder di komputer) dan *graph* (peta jalan atau jaringan pertemanan).

3. Kegunaan ADT

ADT (Abstract Data Type) hanya mendefinisikan *apa* yang bisa dilakukan (operasi dan perilakunya), bukan *bagaimana* cara kerjanya. Satu ADT, misalnya stack, bisa diimplementasikan dengan list atau linked list. Kode yang memakainya tetap sama karena hanya bergantung pada operasi push dan pop. Dengan begitu implementasi bisa diganti sesuai kebutuhan tanpa mengubah program lain.

4. Queue vs stack untuk antrean layanan

Antrean layanan harus adil: yang datang lebih dulu dilayani lebih dulu. Itu prinsip FIFO (First In, First Out) pada queue. Operasinya: enqueue menambah elemen di belakang, dequeue mengambil elemen dari depan. Stack memakai LIFO, jadi yang datang terakhir dilayani lebih dulu dan pelanggan yang datang awal bisa menunggu tanpa batas.

5. Perbedaan pertumbuhan

- **O(1)**: waktu tetap berapa pun ukuran data, misalnya mengakses list[3].
- **O(log n)**: tumbuh sangat lambat karena tiap langkah membuang sekitar separuh data, misalnya binary search. Untuk 1 juta data, sekitar 20 langkah.
- **O(n)**: waktu sebanding dengan ukuran data. Data 10 kali lebih banyak, kerja 10 kali lebih banyak (pencarian linear).
- **O(n^2)**: waktu naik secara kuadrat. Data 10 kali lebih banyak, kerja sekitar 100 kali lebih banyak (loop bersarang).

- **6. Program list dan pencarian linear**

```
angka = [14, 7, 23, 9, 31]


print("Isi list:")
for x in angka:
    print(x)


def cari(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i
    return -1


target = 23
hasil = cari(angka, target)
if hasil != -1:
    print(f"{target} ditemukan pada indeks {hasil}")
else:
    print(f"{target} tidak ditemukan")
```

7. Dictionary lebih sesuai daripada list

Contohnya sistem data mahasiswa yang sering mencari nama berdasarkan NIM. Dengan dictionary, mahasiswa.get('22003') langsung mengakses nilai lewat kunci dan rata-rata cepat (mendekati O(1)). Dengan list, program harus memeriksa elemen satu per satu (O(n)), yang makin lambat saat data bertambah.

8. Menyiapkan Google Colab

1. Buka colab.research.google.com dan masuk dengan akun Google.
2. Pilih **New notebook** (Notebook baru).
3. Klik sel kode, lalu ketik print("Halo, Python!").
4. Jalankan dengan tombol ▶ atau Shift + Enter.
5. Pastikan output Halo, Python! muncul di bawah sel. Itu tanda Colab berhasil menjalankan kode Python pertama.

6. **9. Fungsi cari() dengan minimal 10 elemen**

```
data = [42, 15, 8, 91, 33, 27, 64, 5, 78, 19]


def cari(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i
    return -1


for target in [42, 64, 19, 100]:
    print(f"Target {target} -> indeks {cari(data, target)}")
```

Contoh hasil pencatatan:

Target	Indeks hasil	Elemen diperiksa
42	0	1
64	6	7
19	9	10
100	-1	10

Fungsi harus memeriksa hampir seluruh elemen ketika target berada di posisi paling akhir (19) atau tidak ada di list (100). Pencarian linear baru berhenti saat target ketemu, sehingga pada kasus terburuk semua n elemen diperiksa. Itulah alasan kompleksitasnya O(n).